Release plan CloudGuard

Table of Contents

Introduction

This document contains the release plan for the CloudGuard project. It describes the steps and configurations required to fully start the project and get it to work.

1. Environment Variables

Set environment variables to enable the connection between the backend and the various external services. Create an **.env** file at the root of the backend project, based on the provided **env-vars** file.

```
GOOGLE_CLIENT_EMAIL=google-client-service-account
GOOGLE_PRIVATE_KEY=google-service-account-private-key
TEAMLEADER_CLIENT_ID=teamleader-api-client-id
TEAMLEADER_CLIENT_SECRET=teamleader-api-client-secret
SUPABASE_URL=supabase-project-url
SUPABASE_KEY=supabase-project-secret-key
MAIL_USERNAME=mail-account-username
MAIL_PASSWORD=mail-account-password
BACKEND_PORT=port-for-backend-connection
CLOUDMEN_ADMIN_EMAILS=emails-for-CLOUDMEN-admin-accounts,
```

2. Teamleader Configuration

Add the Teamleader API details to verify the connection.

1. Create an integration/application in the Teamleader Developer Portal.

2. Once the application is created, you will receive a Client ID and a corresponding Client Secret.
3. Enter these details exactly for the variables TEAMLEADER_CLIENT_ID and TEAMLEADER_CLIENT_SECRET in your **.env** file.

3. Supabase Configuration

- Create an account via https://supabase.com/dashboard/sign-up.
- On the platform, choose “start project”, create an organization, choose a password, and click on “create project.”
- Navigate to the API settings page.
- Copy the Project URL and the Service/Secret API-key for the backend integration.
- Paste these values for SUPABASE_URL and SUPABASE_KEY in your **.env** file.

4. Generating & Saving Teamleader Tokens

Save the Teamleader tokens in the database for the API connection.

1. Open the authorization link in your browser to start the authentication flow. Replace client_id= in the URL below with your own Client ID from chapter 2:

   https://focus.teamleader.eu/oauth2/authorize?client_id=je-client-id-van-teamleader&response_type=code&redirect_uri=https://oauth.pstmn.io/v1/callback

2. Grant permission to Teamleader; you will then be redirected to an empty page.

3. Copy the value after code= from the URL of this empty page (this always starts with def...).

4. Now you must fill in all colored fields in the request below:
   a. **Client ID** and **Client Secret** from the previous chapter
   b. **Code** from the previous step

```
curl -X POST https://focus.teamleader.eu/oauth2/access_token \
-d "client_id=jouw-client-id-van-teamleader" \
-d "client_secret=jouw-client-secret-van-teamleader" \
-d "code=jouw-code-van-de-url-van-de-lege-pagina " \
-d "grant_type=authorization_code" \
-d "redirect_uri=https://oauth.pstmn.io/v1/callback"
```

5. Execute the completed request (e.g., in the console) to obtain the access and refresh tokens.

6. Open the SQL Editor in the Supabase side panel.

7. Execute the following SQL code and replace the placeholders with your new access and refresh tokens:

```
INSERT INTO "public"."teamleader_tokens" ("id", "access_token", "refresh_token",
"is_refreshing") VALUES (1, 'jouw_access_token_van_teamleader',
'jouw_refresh_token_van_teamleader', false);
```

5. Google Cloud Configuration (Service Account)

To grant the application access to the Google Workspace environment, an API connection must be made via Google Cloud.

1. Create a Service Account in the Google Cloud Console.

2. Generate a JSON key for this Service Account.

3. Copy the email address from the JSON file and paste it into GOOGLE_CLIENT_EMAIL.

4. Copy the entire private key (including the -----BEGIN PRIVATE KEY----- and -----END PRIVATE KEY----- parts) and paste it into GOOGLE_PRIVATE_KEY.

6. Google Admin Console Configuration (Domain-wide Delegation)

Set up Domain-wide Delegation (DWD) to grant the application access to your organizational data.

1. Log in to the Google Admin Console with a super admin account.

2. Go to **Security → Access and data control → API controls**

3. Click on "MANAGE DOMAIN DELEGATION" at the bottom and add a new API client.

4. Enter the Client ID of the Service Account created in the previous chapter.

5. Add the required OAuth scopes listed below one by one, which determine what data the application has (Read-Only) access to.

```
https://www.googleapis.com/auth/admin.directory.user.readonly
https://www.googleapis.com/auth/admin.directory.rolemanagement.readonly
https://www.googleapis.com/auth/admin.directory.group.readonly
https://www.googleapis.com/auth/admin.directory.group.member.readonly
https://www.googleapis.com/auth/apps.groups.settings
https://www.googleapis.com/auth/cloud-identity.groups.readonly
https://www.googleapis.com/auth/drive.readonly
https://www.googleapis.com/auth/admin.directory.orgunit.readonly
https://www.googleapis.com/auth/admin.directory.device.mobile.readonly
https://www.googleapis.com/auth/cloud-identity.policies.readonly
https://www.googleapis.com/auth/admin.directory.user.security
https://www.googleapis.com/auth/admin.directory.customer.readonly
https://www.googleapis.com/auth/chrome.management.policy.readonly
https://www.googleapis.com/auth/apps.licensing
https://www.googleapis.com/auth/admin.directory.domain.readonly
https://www.googleapis.com/auth/admin.directory.device.chromeos.readonly
```

https://www.googleapis.com/auth/cloud-identity.devices.readonly

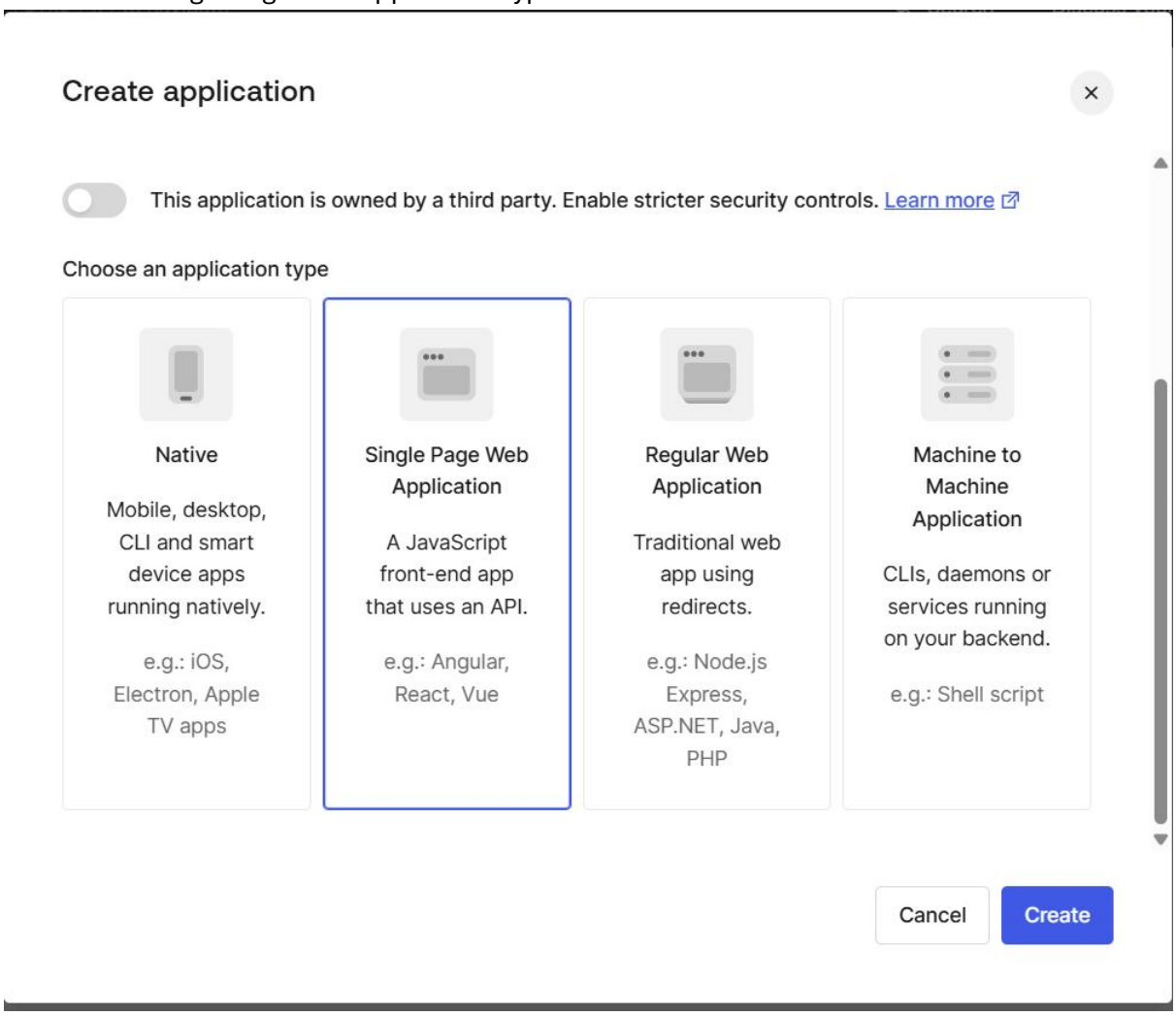
https://www.googleapis.com/auth/admin.reports.usage.readonly

6. Click on “AUTHORIZE” to save the settings.

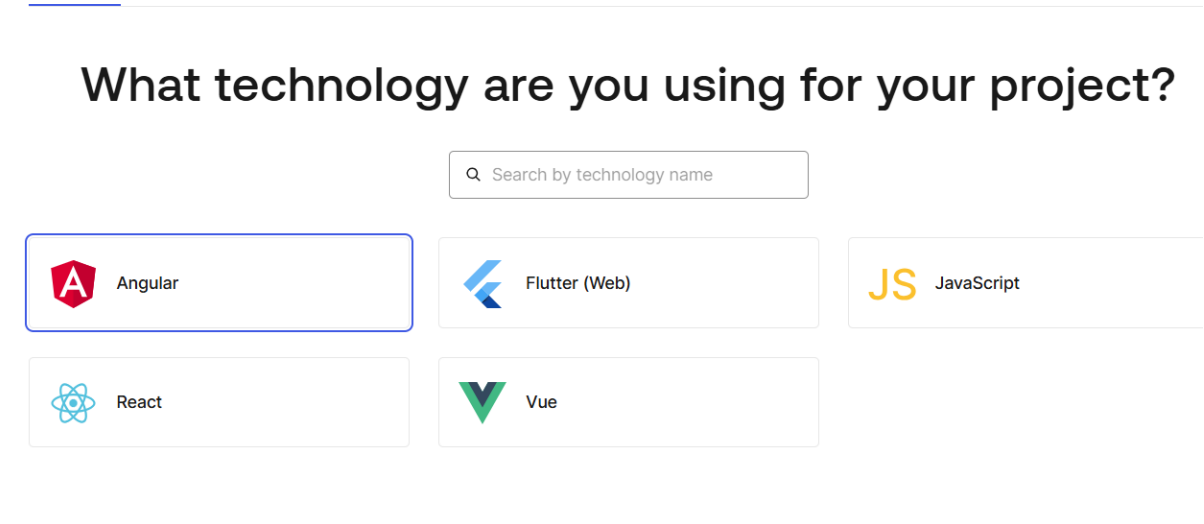
7. Auth0 Configuration

Secure the API and manage user sessions with Auth0.

1. Create an account on Auth0.
2. Go to Applications → Create Application
3. Choose the Single Page Web Application type.



4. Choose the correct technology → Angular

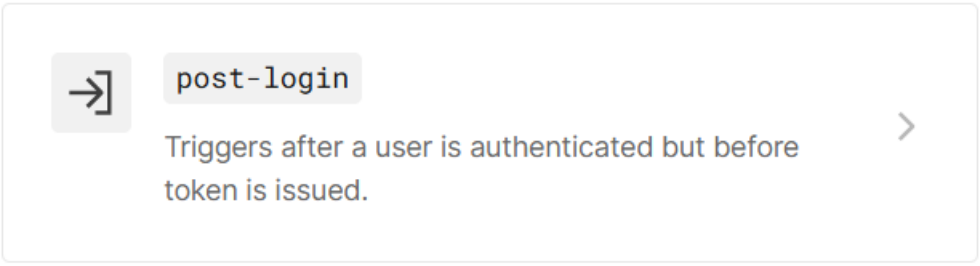


5. Fill in the following fields with the correct values:
   - **Allowed Callback URL:** the link of the website where the Auth0 callback goes to.
   - **Allowed Logout URL:** the link of the website where the Auth0 logout goes to (often the login page).
   - **Allowed Web Origins:** the link of the entire website without any path added to it.

6. Go to the **Settings** tab and copy the following values to **env-vars**:
   - **Domain** (e.g., dev-xxx.eu.auth0.com
   - **Client ID**
   - **Client Secret**

7. Go to the **Action** section.

8. Click on **post-login** and create a new action.



9. Choose **Create Custom Action** and give it a clear name.

10. Replace the entire code field with this exact code:

```
exports.onExecutePostLogin = async (event, api) => {
 if (event.connection.strategy !== 'google-oauth2') return;


 const userEmail = event.user.email;
```

```
 // We geven alleen het e-mailadres mee als veilige claim.
 // De backend bepaalt later pas de rechten (Admin of User).
 api.idToken.setCustomClaim('https://cloudguard.com/workspace_email', userEmail);
 api.accessToken.setCustomClaim('https://cloudguard.com/workspace_email', userEmail);

 console.log(`Succesvolle login via Google Workspace voor: ${userEmail}`);
};
```

11. Click on **Deploy** to save it and put it into operation.

8. E-mail & Admin Configuration

The application uses e-mails for notifications and requires basic settings for administrators.

- **Mail:** Enter the SMTP/login details of the sending e-mail account at MAIL_USERNAME and MAIL_PASSWORD.

- **Administrators:** Enter a comma-separated list of e-mail addresses at CLOUDMEN_ADMIN_EMAILS. These addresses automatically receive administrator access to manage accounts within CloudGuard.

- **Poort:** Define which port the backend should run in and enter this at BACKEND_PORT (the standard is usually 8080).

9. Docker & Traefik Setup

1. Install Docker if you haven't already done so. Check this with the command docker –version. Use the Docker website or the provided script in the Traefik folder for installation.

2. Ensure Docker is running.

3. Create the network for the containers with the command docker network create traefik

4. Ensure the **.env** files in both the root folder and the Traefik folder contain the correct data (as indicated in **env-vars**).

10. Starting Containers with Docker Compose

Once all configurations are complete and Docker is running, you can start the containers.

- Execute the command docker compose up -d --build in the Traefik folder at the root of the project.

- Then execute the same command (docker compose up -d --build) at the root of the project.

- The application will now build and start Traefik, the backend, the frontend, and the database on the configured ports.

Bronnen

Teamleader Developer Portal. (z.d.). http://developer.teamleader.eu

Supabase. (z.d.). Supabase: The Postgres development platform. https://supabase.com

Google Cloud Console. (z.d.). https://console.cloud.google.com

Google Admin Console. (z.d.). https://admin.google.com

Auth0. (z.d.). Auth0: Secure AI Agent & User Authentication. https://auth0.com